SANATH S GOWDA — AI & SYSTEMS ENGINEERING INTERVIEW PLAYBOOK

Complete Architecture, Mathematical Formulas & Interview Q&A Guide for OmniCache & Guard, AutoRepo, and FinSight

1. OMNICACHE & GUARD — Distributed Semantic AI Gateway & Zero-Trust WAF

Live App: https://omnicache-gateway.onrender.com | **GitHub:** https://github.com/Sanathgowda-17/omnicache-gateway | **Local Port:** http://localhost:8000

- **30-Second Elevator Pitch:** "In production AI systems, 30–40% of user prompts are semantically repetitive, costing thousands of dollars and adding 800–2000 ms of latency per call. Furthermore, enterprises cannot send raw user text to external LLMs without scrubbing PII and blocking prompt injections. I built OmniCache & Guard, an OpenAI-compatible AI Gateway (/v1/chat/completions) that intercepts prompts in under 1 ms, scrubs 6 categories of PII, blocks SQLi/Jailbreak payloads via a WAF, and serves semantically similar queries from a Hybrid Vector Cache in ~2.4 ms (a 345x speedup)."
- **How Hybrid Semantic Caching Works (`cache_engine.py`):** Exact-match caches (like Redis `GET/SET`) fail when two users ask the same question with different wording. OmniCache converts normalized text into L2-normalized vectors combining 65% Word Unigram/Bigram TF-IDF and 35% Character Subword N-Grams (3–5 chars) to handle typos, then computes Cosine Similarity: $CosineSim(A, B) = (A \cdot B) / (||A|| \, ||B||)$. If the similarity exceeds threshold $\tau = 0.72$, the cached response is returned immediately.
- **How the Token-Bucket Rate Limiter (`rate_limiter.py`) & Circuit Breaker (`provider_cascade.py`) Work:** Each tenant bucket holds C tokens and refills continuously at rate R tokens/sec in O(1) time: $tokens = min(C, tokens + \Delta t \times R)$. If upstream LLM APIs fail 3 consecutive times, the 3-state Circuit Breaker trips from **CLOSED** → **OPEN** (failing fast to local fallback) and probes recovery via **HALF_OPEN** after a cooldown window.

2. AUTOREPO — Autonomous 4-Agent PR Reviewer, Compiler AST & OWASP Security Linter

GitHub: https://github.com/Sanathgowda-17/autorepo-agent | **Local Port:** http://localhost:8001

- **30-Second Elevator Pitch:** "Naive AI code review tools pass raw source code to a single LLM prompt, which hallucinates line numbers and cannot deterministically compute algorithmic complexity. I engineered AutoRepo, which parses Python code into its native compiler Abstract Syntax Tree (`ast.NodeVisitor`) and orchestrates a 4-Agent Directed Acyclic Graph (DAG): a SecOps SAST Agent for OWASP/CWE vulnerabilities, an AST Complexity Agent for McCabe & Big-O analysis, a Pytest Synthesizer Agent, and a Lead Patch Critic that rewrites vulnerable code into side-by-side hardened patches."
- **How Compiler AST Analysis Works (`ast_engine.py`):** By subclassing `ast.NodeVisitor`, AutoRepo walks `FunctionDef`, `For`, `While`, `If`, and `Try` nodes to compute **McCabe Cyclomatic Complexity**: $M = 1 + branching\ predicates$. It tracks nested loop depth to classify functions into O(1), O(N), O(N^2), or O(N^3), catches database queries (`cursor.execute`) inside loops (**N+1 Query Detector**), and flags mutable default arguments (`def fn(cache=[])`).
- **OWASP Top-10 / CWE Rules & Webhook Security (`agent_dag.py`, `app.py`):** Deterministically catches CWE-89 (SQL Injection via f-strings), CWE-798 (Hardcoded Secrets), CWE-95 (`eval`/`exec` RCE), CWE-78 (OS Command Injection), and CWE-502 (Pickle Deserialization). Incoming GitHub PR webhooks are authenticated via constant-time `hmac.compare_digest` (`HMAC-SHA256`).

3. FINSIGHT — Multimodal Financial (SEC 10-K / Basel III / RBI) & Legal Hybrid RAG

GitHub: https://github.com/Sanathgowda-17/finsight-rag | **Local Port:** http://localhost:8002

- **30-Second Elevator Pitch:** "Standard vector-only RAG fails in financial and legal applications because dense embeddings smooth over exact regulatory identifiers like `CET1`, `DOR.CRE.REC.21`, `Section 9.2`, or `$115.2B`. I built FinSight, an institutional Hybrid RAG platform that runs Sparse Lexical `BM25Okapi` and Dense Vector Search in parallel, fuses their ranks via Reciprocal Rank Fusion (RRF), applies a Cross-Encoder Reranker, extracts embedded Markdown GAAP tables, and verifies every claim using the RAGAS Evaluation Triad."
- **Why Reciprocal Rank Fusion (RRF) is Used (`hybrid_retriever.py`):** BM25 scores (unbounded `0..25+`) and Cosine Similarity (`0.0..1.0`) live on incompatible numerical scales. RRF combines rank positions scale-free: $RRF(d) = (1 - \alpha) / (k + rank_BM25(d)) + \alpha / (k + rank_Dense(d))$. Top candidates are then re-scored jointly with the query by the Cross-Encoder Reranker, achieving 97.8% Recall@3 vs. 74.0% for Naive Dense-Only RAG.

System	Core Algorithms & Data Structures	Key Quantitative Metric	Primary Interview Topic
OmniCache & Guard	TF-IDF Word/Char N-Gram Vectors, Cosine Sim, Token Bucket, Circuit Breaker, Regex WAF/DLP	~2.4 ms cache hit vs ~850 ms LLM (345x speedup)	System Design, Distributed Caching, API Security
AutoRepo	Python `ast.NodeVisitor`, McCabe Complexity (M), 4-Agent DAG, CWE SAST, HMAC-SHA256	<9 ms full AST + SAST + Pytest synthesis	Compilers, Static Analysis, Agentic Orchestration
FinSight RAG	BM25Okapi (k1=1.5, b=0.75), Dense Cosine, Reciprocal Rank Fusion (RRF), Cross-Encoder, RAGAS	97.8% Recall@3, 99.0% RAGAS Faithfulness	Information Retrieval, Hybrid Search, LLM Grounding